ADVANCED JVM, GC, PERFORMANCE & MEMORY

1. Can you explain how the different garbage collectors work in modern Java?

Serial GC uses a single thread and is best for small, simple applications. **Parallel GC** uses multiple threads to clean memory quickly but freezes the app while doing so. **G1 GC** divides memory into small regions and cleans them incrementally to balance speed and responsiveness. **ZGC** and **Shenandoah** are ultra-low latency collectors that clean memory concurrently without stopping the application.

2. What garbage collector does each Java version use by default?

Java 8 uses Parallel GC by default, which prioritizes high throughput (speed). **Java 9** and later versions (including 11, 17, and 21) use **G1 GC** by default. G1 is chosen because it offers a better balance between performance and keeping the application responsive.

3. How do you find and fix memory leak issues in Java applications?

To find a leak, you monitor the **Heap usage**; if it keeps growing and never goes down after GC, you likely have a leak. You then take a **Heap Dump** (a snapshot of memory) and analyze it using tools like **VisualVM** or **Eclipse MAT**. To fix it, you find the object holding onto the data (like a static list) and modify the code to remove those references when they are done.

4. When you see an OutOfMemoryError, how do you figure out the cause?

First, look at the error message (e.g., "Java heap space" vs "Metaspace"). Then, check the application logs to see the **Stack Trace** leading up to the crash. Finally, use a profiling tool to analyze the **Heap Dump** captured at the moment of the crash to see which large objects filled up the RAM.

5. What coding habits help prevent memory leaks in long-running services?

Always close resources like database connections and file streams using **try-with-resources**. Be very careful with **static collections** (like HashMaps), as they act as global storage that never gets cleaned up. Also, make sure to implement equals() and hashCode() correctly if using custom objects as keys in Maps.

JAVA Interview Buddy

6. How would you design a system that keeps GC activity very low?

You should design the system to create fewer temporary objects, perhaps by reusing objects (Object Pooling) for heavy resources. Use **primitive types** (like int) instead of wrapper objects (Integer) where possible to save memory overhead. In extreme cases, you can use **Off-Heap memory** to store data outside the JVM's direct management.

7. What steps would you take to improve memory usage and scalability?

I would identify and remove large, unused objects from the session or cache. I would also implement **horizontal scaling** (adding more servers) rather than just increasing the heap size of one server. Additionally, tuning the **JVM flags** to set the right heap size (-Xmx) ensures the app has enough room to breathe without wasting resources.

8. How do JVM optimizations impact application performance in real projects?

JVM optimizations, specifically from the **JIT Compiler**, make Java code run nearly as fast as native code (like C++). They detect "hot" methods (frequently used code) and rewrite them into machine code for faster execution. This means your application actually gets faster the longer it runs.

9. What performance improvements have you personally implemented in your system?

This is a personal experience question, but a standard answer is: I optimized a slow service by implementing a **caching layer** (like Redis) to reduce database hits. I also analyzed GC logs and switched from Parallel GC to **G1 GC** to reduce random system freezes. Finally, I fixed a memory leak caused by an unclosed file stream that was crashing the server every week.

10. What changed when Java moved from PermGen to Metaspace in Java 8?

PermGen was a fixed-size memory area that often caused OutOfMemoryError if you loaded too many classes. **Metaspace** replaced it and uses native OS memory, allowing it to grow automatically to fit the classes your application needs. This made Java applications much more stable and less prone to crashing during deployment.

JAVA Interview Buddy

11. What are hidden classes introduced in Java 15?

Hidden Classes are classes that are created dynamically at runtime and cannot be used directly by other classes via bytecode. They are designed for frameworks (like Spring) that generate code on the fly. Their main benefit is that they can be **unloaded** (garbage collected) very easily, preventing memory leaks.

12. What is Java Flight Recorder and when do you use it?

Java Flight Recorder (JFR) is a performance monitoring tool built into the JVM that acts like a "black box" on an airplane. It records system events with extremely low overhead (less than 1% performance impact). You use it in production environments to diagnose difficult performance issues or crashes without slowing down the system.

13. What is a GC safepoint and how does the JVM decide when to stop threads there?

A **Safepoint** is a specific point in the code execution where the JVM can safely pause all threads to perform a task (like Garbage Collection). The JVM inserts these checks (usually at method returns or loop exits). When GC is needed, the JVM signals threads to stop at their next available safepoint.

14. What types of optimizations does the JIT compiler perform (like inlining, escape analysis)?

Method Inlining copies the code of a small method directly into the place where it is called, eliminating the performance cost of the method call. **Escape Analysis** checks if an object is only used inside one method; if so, it allocates it on the Stack instead of the Heap to avoid Garbage Collection overhead.

15. What was biased locking, and why was it removed in Java 15?

Biased Locking was an optimization that assumed a lock would usually be accessed by only one specific thread, so it made re-locking cheaper. It was removed because modern CPU architectures made standard locking very fast, so the complexity of maintaining biased locking was no longer worth the small performance gain.

JAVA Interview Buddy

16. What is GraalVM and how can it improve Java performance?

GraalVM is a high-performance JDK that can compile Java applications into **Native Images** (standalone executables). This allows applications to start almost instantly and use very little memory compared to the standard JVM. It is very popular for "Serverless" functions and microservices.

17. How do G1, ZGC, and Shenandoah differ in terms of pause times and guarantees?

G1 aims for a balance, keeping pauses reasonably short but not guaranteeing milliseconds. **ZGC** and **Shenandoah** are "low-latency" collectors designed to keep pause times **under 10ms** (or even <1ms), regardless of how large your Heap is (even terabytes). They do the heavy lifting concurrently while the app runs.

18. How do you tune G1GC for applications that require very low pause times?

You can tune G1GC by setting the flag -XX:MaxGCPauseMillis=200 (or lower) to tell the JVM your target pause time. You can also adjust the **Heap Region Size**. However, setting the target too low might force G1 to run too frequently, which could lower overall throughput.

19. What steps help reduce Stop-The-World pauses in a production system?

To reduce pauses, you should first verify you have enough **Heap memory** allocated. Switching to a low-latency collector like **ZGC** is often the most effective step. Additionally, reducing the rate at which your app creates objects (allocation rate) gives the GC less work to do.

20. What is NUMA-aware memory allocation and why does the JVM use it?

NUMA (Non-Uniform Memory Access) means a CPU processor can access its local RAM faster than RAM attached to other processors. The JVM uses NUMA-aware allocation to store an object in the RAM closest to the specific CPU thread that is using it. This reduces data travel time and improves speed.

21. How do you identify GC thrashing in a running application?

GC Thrashing happens when the JVM spends more time collecting garbage than running your actual application. You identify it by looking at CPU usage (which spikes high) combined with

JAVA Interview Buddy

GC logs showing frequent "Full GCs" that barely free up any memory. It usually means the application needs more RAM immediately.

22. What is memory alignment and what do compressed OOPS do?

The JVM adds padding to objects so they align perfectly in memory (usually multiples of 8 bytes), which makes CPU access faster. **Compressed Oops** is a trick used on 64-bit systems to represent pointers as 32-bit values instead of 64-bit. This saves significant memory space (up to 30GB of Heap) and improves performance.

23. How does escape analysis help the JVM reduce allocations?

Escape Analysis allows the JIT compiler to determine if a new object is used strictly within a single method or thread. If it does not "escape," the JVM can allocate the object on the **Stack** instead of the Heap. When the method finishes, the memory is freed instantly without needing the Garbage Collector.

24. What is scalar replacement and how does the JVM use it?

Scalar Replacement is an optimization linked to Escape Analysis. If an object doesn't escape a method, the JVM breaks the object apart into its individual primitive fields (scalars). It then stores these variables directly in CPU registers or the Stack, completely avoiding the creation of the actual Object container.

JAVA Interview Buddy

MODULE SYSTEM & NEW JAVA FEATURES

1. What were the main changes introduced in Java 11?

Java 11 was a major Long-Term Support (LTS) release that removed the separate JRE and Java EE modules. It introduced a standardized **HTTP Client** for simpler, non-blocking web requests. It also added convenient string methods like .isBlank() and .lines(), and allowed the var keyword to be used inside lambda expressions.

2. What improvements came with Java 17?

Java 17 is an LTS version that finalized major syntax features to reduce boilerplate code. It introduced **Sealed Classes** (to restrict inheritance) and **Records** (for simple data-holding classes). It also added **Pattern Matching for instanceof**, eliminating the need for manual type casting.

3. What new features are available in Java 21?

Java 21 is a game-changing LTS release primarily known for **Virtual Threads**, which allow handling millions of concurrent tasks with very low memory. It also introduced **Sequenced Collections** (like reversed()) to make list manipulation easier. Additionally, it finalized Pattern Matching for switch statements.

4. How do you convert a monolithic application into Java 9 modules?

You start by grouping related packages into logical units and adding a module-info.java file to the root of each unit. You then explicitly specify which packages to **export** (make public) and which other modules to **require** (depend on). It is usually best to migrate from the bottom up, starting with the utility libraries first.

5. How do modules change the overall architecture of a system?

Modules enforce **strong encapsulation**, preventing external code from accessing internal classes unless explicitly allowed. This creates a cleaner architecture where dependencies are clearly defined, reducing the "spaghetti code" problem. It forces the system to be built as a set of interacting, independent components.

6. What important changes were introduced in Java 9?

The most massive change was the **Java Platform Module System (JPMS)**, which reorganized the JDK itself into modules. It also introduced **JShell**, an interactive tool (REPL) for testing code snippets instantly. Additionally, it allowed private methods inside interfaces to share code between default methods.

7. How do Java modules affect large-scale application design?

Modules solve "dependency hell" by verifying that all required libraries are present at startup, not halfway through execution. They force developers to define clear boundaries, making large applications easier to maintain and refactor. This prevents accidental usage of internal APIs that might change in the future.

JAVA Interview Buddy

8. What is a record in Java and why do we use it?

A **Record** is a special class designed solely to hold immutable data. By simply writing record User(String name, int id) {}, Java automatically generates the constructor, getters, equals(), hashCode(), and toString() methods. We use them to eliminate the huge amount of boilerplate code needed for simple data-carrying classes.

9. What is a sealed class and what problem does it solve?

A **Sealed Class** allows you to strictly control which specific classes are allowed to extend it (using the permits keyword). It solves the problem of uncontrolled inheritance, ensuring that your class hierarchy remains finite and predictable. This is extremely useful for domain modeling and security.

10. What is jlink and how can you build a custom runtime image?

jlink is a tool that allows you to create a custom, lightweight Java Runtime Environment (JRE) containing *only* the modules your application needs. Instead of shipping the heavy, full JDK, you can ship a tiny, optimized runtime image. This is perfect for microservices and containerized deployments (like Docker).

11. How do modules change the behavior of the classloader hierarchy?

In the module system, the ClassLoader no longer simply searches a flat "classpath" linearly. Instead, it uses a **module path** where classes are grouped by module. Each ClassLoader only loads classes from the modules that are explicitly required, preventing conflicts where two libraries contain the same class name.

12. What does “strong encapsulation” mean in the module system?

Strong encapsulation means that a module hides *everything* inside it by default. Even public classes in a package are not accessible to other modules unless that specific package is explicitly **exported** in module-info.java. This prevents external code from relying on internal implementation details.

JAVA Interview Buddy

13. Why was the Nashorn JavaScript engine removed, and what replaced it?

Nashorn was removed because it was difficult to maintain and could not keep up with the rapidly changing ECMAScript (JavaScript) standards. It has been replaced by **GraalVM**, a modern, high-performance polyglot virtual machine that supports JavaScript (and other languages) much more efficiently.

14. How does pattern matching for instanceof work in Java 17?

It allows you to check the type and cast it in a single step. Instead of writing if (obj instanceof String) { String s = (String) obj; }, you simply write if (obj instanceof String s). The variable s is automatically created and ready to use inside the if block, making code cleaner.

15. What are virtual threads (Project Loom) and how do they change concurrency?

Virtual Threads are lightweight threads managed by the JVM rather than the operating system. While creating a standard OS thread is expensive (limiting you to thousands), you can easily create **millions** of virtual threads. This radically simplifies writing high-throughput concurrent applications without complex reactive code.

JAVA Interview Buddy

ADVANCED CONCURRENCY, EXECUTORS & PARALLELISM

1. How does ThreadPoolExecutor work internally?

When a task is submitted, the executor first checks if it has fewer than the **core pool size** of threads; if so, it creates a new thread. If the core pool is full, it puts the task into the **blocking queue**. If the queue is full, it creates new threads up to the **maximum pool size**; if that is also full, it rejects the task.

2. How does an executor check which threads are alive or dead?

The executor wraps every task in a worker object that tracks the thread's lifecycle. If a thread throws an exception or finishes its task naturally, the executor detects that the worker has exited. If the pool size drops below the required core size, the executor automatically starts a new thread to replace the dead one.

3. How can you ensure atomicity without using synchronized?

You can use **Atomic classes** (like AtomicInteger or AtomicReference) from the java.util.concurrent.atomic package. These classes use a hardware-level technique called **CAS (Compare-And-Swap)** to update values safely. This allows for thread-safe updates without the performance cost of stopping threads with heavy locks.

4. What is the difference between visibility and atomicity?

Visibility guarantees that if one thread changes a value, other threads see that change immediately (handled by volatile). **Atomicity** guarantees that a series of steps (like "read, increment, write") happens as a single, unbreakable unit (handled by synchronized or Atomic classes). You often need both for correct concurrency.

5. How do parallel streams work behind the scenes?

Parallel streams rely on the **ForkJoinPool**, specifically the common static pool available to the entire JVM. The data is split into small chunks using a **Spliterator**, and these chunks are processed by worker threads that use **work-stealing** to balance the load. Once processed, the results are combined back into a final result.

6. When should you use volatile instead of synchronization?

Use volatile when you only need to ensure **visibility** for a simple flag or variable (e.g., a boolean running = true flag). Do not use it when you need to perform compound operations like counter++ (read-modify-write), because volatile cannot prevent race conditions in those cases; use synchronization instead.

7. Can a single thread cause a deadlock?

Yes, a single thread can cause a deadlock if it tries to acquire a lock it already holds, but the lock is **non-reentrant** (it doesn't allow re-entry). It can also happen if a thread submits a task to a thread pool and then waits for that task to finish, but the pool is full and the task is stuck in the queue waiting for the thread to become free.

JAVA Interview Buddy

8. What is a deadlock and how do you avoid it?

A **deadlock** happens when two threads are stuck forever because each is waiting for a lock the other holds. To avoid it, you should enforce a strict **ordering of locks** (e.g., always acquire Lock A before Lock B). You can also use tryLock() with a timeout to give up waiting if the lock isn't available.

9. How do you detect deadlocks or thread starvation in an application?

For deadlocks, you can use tools like **jstack** or **VisualVM** to take a thread dump; they explicitly report "Found one Java-level deadlock." For starvation, you must monitor application logs and metrics to see if certain tasks are taking unusually long to start or if specific threads are stuck in a "WAITING" state for extended periods.

10. What design decisions help avoid deadlocks?

The most effective design is to **minimize the scope** of locks, hold them for the shortest time possible. Avoid calling external methods while holding a lock (alien methods). Also, try to use high-level concurrency utilities (like ExecutorService or ConcurrentHashMap) instead of manually managing explicit locks.

11. How does an executor react when a task is interrupted?

When a task is interrupted, the thread executing it usually catches an InterruptedException. The executor itself doesn't simply stop; the specific task code must catch the exception and decide to stop gracefully (e.g., by breaking a loop). If the exception bubbles up, the thread might die, and the executor will create a new one to replace it.

12. What are the different ways to achieve synchronization in Java?

You can use the synchronized keyword (on methods or blocks) or **ReentrantLock** for explicit locking. For lighter synchronization, you can use volatile variables for visibility or **Atomic classes** for simple counters. For collections, you can use concurrent wrappers like Collections.synchronizedList or dedicated classes like ConcurrentHashMap.

JAVA Interview Buddy

13. How is a synchronized method different from a synchronized block?

A **synchronized method** locks the entire object instance (this) or the class (if static) for the duration of the method. A **synchronized block** is more flexible; it allows you to lock only a critical section of code and lets you choose a specific object to lock on. Blocks are generally preferred for performance to keep the "locked" time short.

14. How do CountDownLatch and CyclicBarrier differ?

A **CountDownLatch** is a one-time use counter; threads wait until the count reaches zero, and it cannot be reset. A **CyclicBarrier** is a reusable meeting point; threads wait until a specific number of parties arrive, and then the barrier "trips," resets, and can be used again for the next cycle.

15. What is lock-free programming and how does CAS help achieve it?

Lock-free programming ensures that threads never strictly block each other; if one thread is suspended, others can still make progress. It relies on **CAS (Compare-And-Swap)**, which allows a thread to try updating a variable and retry if it fails, rather than sleeping and waiting for a lock to open.

16. What is the ABA problem and how can AtomicStampedReference fix it?

The **ABA problem** occurs when a variable changes from A to B and back to A; a CAS check sees "A" and thinks nothing changed, causing logic errors. **AtomicStampedReference** fixes this by attaching a version number (stamp) to the reference. Even if the value is back to "A," the stamp will differ, ensuring the CAS correctly detects the change.

17. What is false sharing and why is it bad for multicore performance?

False sharing happens when two independent variables (used by different threads) sit so close in memory that they share the same "cache line." When one thread updates its variable, the CPU invalidates the whole cache line, forcing the other thread to reload its data unnecessarily. This degrades performance significantly.

JAVA Interview Buddy

18. What are memory fences and how do they relate to happens-before rules?

Memory fences (or barriers) are low-level CPU instructions that prevent the processor from reordering instructions across the fence. They ensure that data written before the fence is visible to other threads after the fence. Java's volatile and synchronized keywords automatically insert these fences to enforce **happens-before** guarantees.

19. How does ForkJoinPool operate internally?

ForkJoinPool uses a "divide and conquer" approach where tasks are recursively split into smaller subtasks. Unlike a standard pool where all threads share one queue, each thread in a ForkJoinPool has its own double-ended queue (deque) of tasks. This design minimizes contention between threads.

20. How does the work-stealing algorithm function in Java?

In **work-stealing**, if a thread runs out of tasks in its own queue, it looks at the queues of other busy threads. It "steals" a task from the **tail** (end) of another thread's deque. This ensures that all threads stay busy without fighting over the same tasks at the head of the queue.

21. What is the difference between a Mutex, Semaphore, and ReentrantLock?

A **Mutex** allows only one thread to access a resource at a time (like a lock). A **Semaphore** maintains a set of "permits," allowing a specific number of threads (e.g., 5 connections) to access a resource simultaneously. **ReentrantLock** is a Java class that implements a Mutex but adds advanced features like fairness and interruptibility.

22. How do you find deadlocked threads using jstack?

You run the command jstack <pid> (where pid is the Process ID of your Java app) in the terminal. The output will show the stack trace of all threads. At the very bottom, it specifically looks for patterns and will print **"Found one Java-level deadlock"** listing exactly which threads are involved.

23. How do thread park() and unpark() work?

These are low-level primitives in the LockSupport class. **park()** puts the current thread to sleep (suspends it) efficiently without needing a synchronized block. **unpark(Thread t)** is called by

JAVA Interview Buddy

another thread to wake that specific thread up. Java's high-level Locks use these internally to manage waiting threads.

24. What is a Phaser and when would you use it?

A **Phaser** is a flexible synchronization barrier, similar to CyclicBarrier but more dynamic. It allows the number of registered parties (threads) to verify and change over time. You use it for multi-phase algorithms where threads might finish early or new threads might join midway through the process.

25. How are virtual threads different from platform threads in Java 21?

Platform threads are essentially OS kernel threads; they are heavy, expensive to create, and limited in number. **Virtual threads** are managed entirely by the JVM; they are extremely lightweight, so you can create millions of them. Virtual threads unmount from the CPU when waiting for I/O, making them incredibly efficient for high-throughput server applications.

JAVA Interview Buddy

ARCHITECTURE, DESIGN APPROACH & LARGE-SCALE DECISIONS

1. How do you secure a system that loads plugins at runtime?

You should run plugins in a restricted environment, often called a **sandbox**, with limited permissions. Use a dedicated SecurityManager (or similar isolation logic) to prevent the plugin from accessing the file system or network. Additionally, load each plugin in its own separate ClassLoader so it cannot access or modify the core application's internal classes.

2. How do you safely load plugins using a custom ClassLoader?

You create a custom ClassLoader that overrides the findClass() method to load the plugin's .jar file from a specific directory. Crucially, you should ensure the parent loader cannot see these classes, creating strict isolation. This prevents "DLL Hell" (version conflicts) where a plugin might use a different version of a library than the main app.

3. How would you design a payment module using interfaces or abstract classes?

I would define a common interface named PaymentProcessor with a method like process(amount). specific implementations, such as CreditCardProcessor or PayPalProcessor, would implement this interface with their specific logic. This allows the main system to treat all payment methods identically without knowing the details of each one.

4. How do you use the Builder pattern for complex configuration objects?

Instead of a constructor with 20 confusing parameters, I would use a static inner Builder class. You call readable methods like .setTimeout(5000) and .setRetryCount(3) one by one. Finally, you call .build() to validate the settings and return the immutable configuration object.

5. How would you fix poor OOP design in a system like a Vehicle example?

If I see a deep inheritance tree like Vehicle -> Car -> ElectricCar, which is rigid, I would refactor it using **Composition**. I would create a Vehicle class that *contains* independent components like Engine, Wheels, and FuelType. This allows me to mix and match features easily without breaking a fragile hierarchy.

6. How do you choose the right collection for high-frequency trading systems?

In HFT, standard collections create too much garbage (object overhead), causing slow GC pauses. I would use **primitive arrays** or specialized libraries (like Agrona or HPPC) that store int and long data directly in memory. This avoids "boxing" (converting primitives to objects) and keeps the system extremely fast.

7. How do you prevent cache-related memory leaks?

Use a **WeakHashMap**, where entries are automatically deleted if the key is no longer used by the rest of the application. Alternatively, use a robust caching library like **Caffeine** or **Guava** and configure a "Time-To-Live" (TTL) or a maximum size limit. This ensures old entries are evicted before the memory fills up.

JAVA Interview Buddy

8. How would you design a system where many threads share the same resource?

I would use a **ReadWriteLock** to optimize performance. This allows multiple threads to *read* the data simultaneously without blocking each other, which is very fast. The lock only blocks other threads when a specific thread needs to *write* or update the data, ensuring safety.

9. How do you build your own thread-safe HashMap without using ConcurrentHashMap?

I would create an array of "buckets" (linked lists) to store data. To ensure thread safety, I would use synchronized blocks, but instead of locking the entire map, I would lock only the specific bucket being accessed (Lock Striping). This mimics the internal logic of ConcurrentHashMap to allow concurrent access.

10. How can you build an LRU cache with LinkedList or LinkedHashMap?

The easiest way is to extend Java's LinkedHashMap and override the removeEldestEntry() method. Inside that method, you check if the map size exceeds your limit (e.g., 100 items); if true, it automatically deletes the oldest item. You must also set the "accessOrder" flag to true in the constructor so accessed items move to the end of the list.

11. How do LinkedHashSet and TreeSet compare in performance?

LinkedHashSet is significantly faster (O(1)) for adding and finding items because it relies on a hash table. **TreeSet** is slower (O(log n)) because it has to sort elements into a tree structure every time you add one. Use LinkedHashSet if you just need insertion order; use TreeSet only if you need the data sorted by value.

12. How can the JVM help reduce an application’s memory footprint?

You can enable **Compressed Oops** (-XX:+UseCompressedOops) to use 32-bit pointers on 64-bit systems, which saves significant space. You can also enable **String Deduplication**, which tells the Garbage Collector to identify duplicate strings and make them share the same character array.

JAVA Interview Buddy

13. How would you explain the JVM lifecycle to a new developer?

The JVM starts when you run the java command, which initializes the Bootstrap ClassLoader. It then finds your main class, loads it, and executes the public static void main method. The JVM remains alive as long as any non-daemon thread is running; once the last thread finishes, the JVM shuts down.

14. How do you design good versioning for a large REST API?

The most common and clear approach is **URI Versioning** (e.g., /api/v1/users). This makes it obvious which version a client is using just by looking at the URL. Another option is **Header Versioning**, where the client sends a custom header requesting a specific version, keeping the URL cleaner but harder to test in a browser.

15. How do you maintain backward compatibility in a distributed system?

Always follow the rule of **additive changes**: only add new fields or endpoints; never delete or rename existing ones that clients rely on. If a breaking change is necessary, support both the old and new versions simultaneously for a transition period. This gives clients time to migrate without their application crashing.

16. How do you design a rate limiter (token bucket or leaky bucket)?

I would use the **Token Bucket** algorithm. Imagine a bucket that gets filled with tokens at a constant rate (e.g., 10 tokens/second). Each request must take a token to proceed; if the bucket is empty, the request is rejected. This allows for short bursts of traffic while maintaining a steady average limit.

17. How would you design a high-throughput logging system?

I would use **Asynchronous Logging** (like Log4j2 Async Appender). The main application thread simply places the log message into a memory queue and returns immediately, keeping the app fast. A separate background thread picks up messages from the queue and writes them to the disk in batches.

JAVA Interview Buddy

18. How do you build distributed locking safely?

I would use a centralized external store like **Redis** (with the Redlock algorithm) or **ZooKeeper**. A service tries to write a unique key with a timeout (TTL) to this central store. If successful, it holds the lock; other services see the key exists and wait until it expires or is deleted.

19. When do you choose eventual consistency instead of strong consistency?

Choose **Strong Consistency** (like banking) when every read *must* show the absolute latest data, even if it makes the system slower. Choose **Eventual Consistency** (like social media likes) when high availability and speed are more important. In eventual consistency, it's okay if different users see slightly different data for a few seconds.

20. How do you stop cascading failures in a microservice system?

Implement the **Circuit Breaker** pattern. If a specific service fails repeatedly (e.g., 5 times in a row), the circuit "opens" and immediately fails all future requests without trying to reach the service. This prevents waiting threads from piling up and crashing the rest of the system.

21. What architectural differences exist between microservices, monoliths, and modular monoliths?

A **Monolith** puts all code in one deployable unit; it is simple but hard to scale. **Microservices** split the app into many independent network services; it scales well but is very complex to manage. A **Modular Monolith** is a middle ground: it is one deployable unit, but the code is strictly separated into logical modules internally.

JAVA Interview Buddy

22. How do you make your API operations idempotent?

To make an operation **idempotent** (safe to repeat), the client sends a unique ID (Idempotency Key) with the request. The server checks if it has already processed a request with that ID. If it has, it returns the previous cached response instead of executing the action a second time.

23. How do you handle retries and backoff in a network-based system?

When a network call fails, you should not retry immediately to avoid overwhelming the server. Use **Exponential Backoff**: wait 1 second, then 2, then 4, then 8 seconds. This prevents

"thundering herd" problems where a recovering server is instantly crashed again by thousands of retry attempts.

SECURITY, NETWORKING & HARDENING

1. What security rules apply when code is loaded from remote locations?

When loading remote code, you must verify its source using **Digital Signatures** to ensure it hasn't been tampered with. Historically, Java used a **Security Manager** to run such code in a restricted "sandbox" with limited permissions (no file/network access). In modern systems, it is safer to run untrusted code in a completely separate, isolated container (like Docker) to prevent it from harming the host system.

2. How do you protect a system from DDoS attacks?

To stop Distributed Denial of Service (DDoS) attacks, you should use a **Load Balancer** and a Content Delivery Network (CDN) to spread traffic across many servers so no single machine gets overwhelmed. Implementing **Rate Limiting** ensures that no single user can send thousands of requests per second. Additionally, a Web Application Firewall (WAF) can help detect and block malicious traffic patterns before they reach your app.

3. What is the difference between REST and RESTful?

REST (Representational State Transfer) is the set of architectural **rules** and theories for designing network applications. **RESTful** is an adjective used to describe an actual web API that correctly follows those rules. Ideally, REST is the "grammar," and a RESTful service is a "correct sentence" written using that grammar.

4. What was the Java Security Manager and why is it deprecated now?

The **Security Manager** was a tool that allowed applications to run untrusted code in a restricted environment (sandbox) by checking permissions for every system operation. It is deprecated in newer Java versions because it was difficult to configure correctly and had a significant performance cost. Modern security relies on isolating applications at the OS or container level instead.

JAVA Interview Buddy

5. How do you protect your application from deserialization attacks?

The best protection is to avoid deserializing data from untrusted sources entirely. If you must use it, use a **whitelist** (via ObjectInputFilter) to strictly allow only safe, expected classes to be loaded. Ideally, switch to safer data formats like **JSON** or **XML** instead of using Java's native serialization.

6. What is CSRF and how do you prevent it in Java?

CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks a user's browser into sending unwanted actions to a site where the user is logged in. To prevent it, the server should generate a unique **Anti-CSRF Token** for every session and require it to be present in every form submission. If the token is missing or incorrect, the server rejects the request.

7. What is SQL injection and how do you stop it?

SQL Injection happens when an attacker types malicious SQL code into an input field (like a login box) to trick the database into revealing or deleting data. To stop it, always use **PreparedStatements** (parameterized queries) instead of concatenating strings. This ensures the database treats user input strictly as data, not executable code.

8. Can you explain the HTTPS handshake in simple terms?

The HTTPS handshake is a conversation where the client and server introduce themselves and agree on a secret code. First, the server shows its ID card (**SSL Certificate**) to prove it is legitimate. Then, they use complex math to securely create a unique **session key** that they will use to lock (encrypt) all messages sent between them.

9. What is JWT and how do you secure it properly?

JWT (JSON Web Token) is a compact token used to safely prove a user's identity without keeping a session on the server. To secure it, you must sign it with a strong, secret algorithm so it cannot be faked. Also, always send it over **HTTPS** and never store sensitive secrets (like passwords) inside the token payload, as it can be read by anyone.

JAVA Interview Buddy

REFLECTION, PROXIES & DEPENDENCY INJECTION

1. How do dynamic proxies work in Java?

Java's Proxy class creates a brand new class at runtime that implements a specific set of **interfaces**. When you call a method on this proxy object, the call is intercepted and routed to a designated InvocationHandler. This handler allows you to execute custom logic (like logging or checking security) before or after passing the call to the real object.

2. How does dependency injection based on reflection work?

Frameworks like Spring use **Reflection** to scan your classes for annotations (like @Autowired). Even if your fields or constructors are private, the framework uses setAccessible(true) to bypass Java's access controls. It then manually inserts (injects) the required dependency objects directly into those fields at runtime.

3. Why would you make a class final to prevent extension?

You make a class final to guarantee that its behavior cannot be changed or overridden by a subclass. This is essential for security and for creating **Immutable** objects (like the String class). If a class cannot be extended, you can be 100% sure that the code running is exactly what you wrote, not a modified version from a subclass.

4. What is MethodHandle and what does invokedynamic do?

MethodHandle is a modern, faster alternative to standard Reflection that acts like a direct, typed reference to a method. invokedynamic is a special bytecode instruction that allows the JVM to delay linking a method call until the moment it is actually executed. Together, they are the "magic" that makes Java **Lambda expressions** efficient and flexible.

5. How does Spring use reflection and proxies internally?

Spring uses **Reflection** to instantiate your Beans and inject dependencies by inspecting private fields and constructors. It uses **Proxies** to handle "Cross-Cutting Concerns" like Transactions or Security. When you call a method on a Spring Bean, you are actually calling the Proxy, which starts the transaction, runs your method, and then commits the transaction.

JAVA Interview Buddy

6. What is the difference between JDK dynamic proxies and CGLIB proxies?

JDK Dynamic Proxies are built into Java but can only create proxies for classes that implement an **interface**. **CGLIB (Code Generation Library)** is a third-party library that is more powerful; it creates proxies for classes *without* interfaces by dynamically generating a **subclass** of your target class.

7. How does Hibernate use reflection and bytecode enhancement?

Hibernate uses **Reflection** to read from and write to your private entity fields to map them to database columns. It uses **Bytecode Enhancement** (via CGLIB or Javassist) to create proxy versions of your entities. These proxies detect when you modify a field (Dirty Checking) or fetch data from the database only when you try to access it (Lazy Loading).

ENTERPRISE DEBUGGING, SCALING & REAL PRODUCTION ISSUES

1. How do you track down and fix memory leaks in production?

First, I monitor the application to see if **Heap usage** is constantly increasing without dropping. If confirmed, I take a **Heap Dump** (a snapshot of memory) and analyze it using a tool like **Eclipse Memory Analyzer (MAT)**. I look for objects that are consuming the most space, usually large static collections and fix the code to ensure they are cleared properly.

2. What performance improvements did you personally add in past projects?

This is a personal experience question, but a standard answer is: I optimized a slow API endpoint by implementing **Redis caching**, which reduced the response time from 2 seconds to 200ms. I also identified a "N+1" query problem in Hibernate and fixed it using JOIN FETCH, which significantly reduced the load on our database.

3. How do equals() and hashCode() impact caching?

Caches (like HashMap) rely on hashCode() to determine which memory bucket to look in, and equals() to confirm it found the correct key. If these methods are not implemented correctly, the cache might fail to retrieve existing data or store duplicate entries. This leads to cache misses and unnecessary memory waste.

JAVA Interview Buddy

4. What steps help improve scalability and memory usage?

To improve scalability, I design systems to be **stateless** so we can easily add more servers (horizontal scaling). To improve memory usage, I avoid creating unnecessary temporary objects and use efficient data structures (like primitive arrays) where possible. I also ensure caching is used effectively to reduce redundant processing.

5. How would you debug a NullPointerException happening on a UI action?

I would look at the **stack trace** in the logs to identify the exact line of code where the crash happened. I would then examine the variables on that line to see which one was null when it shouldn't have been. To fix it, I would add a null check or wrap the value in an Optional.

6. How do you investigate a CPU spike in production using tools like top, jstack, or profilers?

I use the top -H command to find the specific **thread ID** that is consuming the most CPU. I convert that ID to hexadecimal format. Then, I capture a thread dump using jstack and search for that hex ID to see exactly which method or loop that thread is currently executing.

7. How do you know when your thread pool is overloaded or saturated?

You can monitor the thread pool's **queue size**; if it is constantly full or growing, the pool is overloaded. You might also see RejectedExecutionException in your logs, which means the pool cannot accept any new tasks. High latency for simple tasks is another common sign.

8. Which profiling tools do you use in production (e.g., JFR, Mission Control, YourKit)?

I prefer using **Java Flight Recorder (JFR)** because it is built into the JVM and has extremely low performance overhead (less than 1%). I then analyze the recording files using **JDK Mission Control (JMC)** to visualize CPU usage, memory allocation, and garbage collection pauses.

9. How do you detect GC pressure using garbage collection logs?

I look for frequent **"Full GC"** events in the logs that take a long time to run but barely free up any memory. If the logs show that the application is spending more than 5-10% of its time just doing Garbage Collection, it is suffering from high GC pressure.

JAVA Interview Buddy

10. How do you investigate slow database queries from the Java side?

I use Application Performance Monitoring (APM) tools like **New Relic** or **Datadog** to pinpoint which SQL queries are taking the longest. In development, I enable Hibernate statistics or use a proxy like P6Spy to log the execution time of every SQL statement generated by the application.

11. What is backpressure and how do you implement it?

Backpressure is a mechanism where a slow consumer tells a fast producer to "slow down" so it doesn't get overwhelmed and crash. It is a key concept in **Reactive Programming**. In Java, you implement it using libraries like **Project Reactor** or **RxJava**, which handle the flow control automatically.

JAVA Interview Buddy